Project Title

Superstore Sales Data Analysis Using Pandas

Problem Statement

- Retail businesses generate large volumes of transactional data daily. Analyzing this data manually is inefficient and time-consuming. Businesses need efficient tools to clean, process, and analyze sales data in order to identify trends, improve profitability, and support data-driven decision making.
- In this project, we use Pandas, a powerful Python library for data manipulation and analysis, to explore and clean a retail sales dataset. The dataset contains inconsistencies such as missing values, duplicate records, and inconsistent text formatting. Using Pandas, we will clean the data, perform analysis, and extract meaningful business insights.

Objectives

The main objectives of this project are:

- To understand how Pandas handles real-world datasets
- To perform data cleaning and preprocessing
- To explore dataset structure and statistical properties
- To analyze sales and profit performance
- To identify patterns and trends in the retail dataset
- To generate business insights from sales data

Dataset Description

- The dataset contains retail sales transaction records including information about orders, products, sales values, and profits.

The dataset intentionally contains data quality issues such as:

- Missing values
- Duplicate rows
- Inconsistent category names

These issues will be addressed during data cleaning.

Step 1 Basics And Dataset Understanding

Import Required Libraries

In [285…

```
import pandas as pd
import numpy as np
```

Explanation:

- Pandas → Data manipulation and analysis
- NumPy → Numerical operations

Load the Dataset

In [286…

```
df = pd.read_csv("Sales_Data.csv")

df.head()
```

Out[286…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quantity
0	ORD-10000	2022-07-14	South	Houston	Office Supplies	Binders	574.32	
1	ORD-10001	2022-06-07	South	Houston	Technology	Accessories	1427.06	
2	ORD-10002	2022-05-21	West	Houston	furniture	Binders	1103.35	
3	ORD-10003	2022-10-02	east	New York	Furniture	Binders	906.01	
4	ORD-10004	2022-01-07	east	San Antonio	Office Supplies	Accessories	250.91	

Explanation:

- read_csv() loads the dataset
- head() displays the first 5 rows

Display Last Rows

In [287…

```
df.tail()
```

Out[287…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quan
2045	ORD-10360	2022-02-23	east	San Jose	furniture	Phones	594.49	
2046	ORD-10282	2022-10-28	South	Los Angeles	Furniture	Machines	575.86	
2047	ORD-11502	2022-09-12	East	Houston	Technology	Tables	57.95	
2048	ORD-11279	2022-05-24	East	NaN	Technology	Accessories	758.35	
2049	ORD-11958	2022-12-05	East	Phoenix	Technology	Machines	961.54	

Display Random Sample

In [288…

```
df.sample(5)
```

Out[288…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quar
431	ORD-10431	2022-12-25	east	San Jose	Furniture	Tables	1147.04	
689	ORD-10689	2022-07-03	east	San Jose	Furniture	Machines	296.82	
1519	ORD-11519	2022-12-14	WEST	San Jose	Furniture	Machines	1392.01	
139	ORD-10139	2022-12-19	east	San Jose	Office Supplies	Phones	1458.24	
1940	ORD-11940	2022-01-08	South	San Antonio	Technology	Machines	106.66	

Explanation:

- Displays random rows from dataset.
- Useful to understand data distribution.

Check Dataset shape

In [289…

```
df.shape
```

Out[289…

```
(2050, 10)
```

- 2050 rows
- 10 columns

Display Column Names

- Shows all dataset columns.

In [290…

```
df.columns
```

Out[290…

```
Index(['Order_ID', 'Order_Date', 'Region', 'City', 'Category', 'Sub_Category',
       'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='object')
```

Check Data Types

In [291…

```
df.dtypes
```

Out[291…

```
Order_ID         object
Order_Date       object
Region           object
City             object
Category         object
Sub_Category     object
Sales           float64
Quantity          int64
Discount        float64
Profit          float64
dtype: object
```

Dataset Information

In [292…

```
df.info
```

Out[292…

```
<bound method DataFrame.info of        Order_ID  Order_Date Region         City
Category Sub_Category  \
0     ORD-10000  2022-07-14  South      Houston  Office Supplies      Binders
1     ORD-10001  2022-06-07  South      Houston       Technology  Accessories
2     ORD-10002  2022-05-21   West      Houston        furniture      Binders
3     ORD-10003  2022-10-02   east     New York        Furniture      Binders
4     ORD-10004  2022-01-07   east  San Antonio  Office Supplies  Accessories
...         ...         ...    ...          ...              ...          ...
2045  ORD-10360  2022-02-23   east     San Jose        furniture       Phones
2046  ORD-10282  2022-10-28  South  Los Angeles        Furniture     Machines
2047  ORD-11502  2022-09-12   East      Houston       Technology       Tables
2048  ORD-11279  2022-05-24   East          NaN       Technology  Accessories
2049  ORD-11958  2022-12-05   East      Phoenix       Technology     Machines

        Sales  Quantity  Discount  Profit
0      574.32         7       0.0   90.93
1     1427.06         3       0.1  468.06
2     1103.35         7       NaN   58.24
3      906.01         8       0.0  247.61
4      250.91         6       0.2   -9.41
...       ...       ...       ...     ...
2045   594.49         8       0.2  -69.66
2046   575.86         2       0.3     NaN
2047    57.95         2       0.2    8.19
2048   758.35         1       0.3  152.68
2049   961.54         3       0.2  -97.67

[2050 rows x 10 columns]>
```

This shows:

- number of non-null values
- column data types
- memory usage

This is important for data cleaning.

Statistical Summary

In [293…

```
df.describe()
```

Out[293...

	Sales	Quantity	Discount	Profit
count	2010.000000	2050.000000	1625.000000	2006.000000
mean	757.668791	4.996098	0.149969	112.245344
std	432.618401	2.601570	0.111369	166.224063
min	24.760000	1.000000	0.000000	-208.010000
25%	374.130000	3.000000	0.100000	0.157500
50%	768.690000	5.000000	0.100000	67.085000
75%	1131.195000	7.000000	0.300000	205.300000
max	1499.580000	9.000000	0.300000	669.360000

Shows statistical metrics for numerical columns:

- mean
- standard deviation
- min
- max
- quartiles

Step 2 Data Cleaning

Check Missing Values

In [294...

```
df.isnull().sum()
```

Out[294...

```
Order_ID          0
Order_Date        0
Region            0
City             43
Category          0
Sub_Category      0
Sales            40
Quantity          0
Discount        425
Profit           44
dtype: int64
```

Initial Insight

From the initial dataset exploration, we observe that the dataset contains several columns related to sales transactions. However, the dataset also contains missing values and inconsistent entries which must be cleaned before performing detailed analysis.

Display Rows with Missing Values

In [295…

```
df[df.isnull().any(axis=1)]
```

Out[295…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
2	ORD-10002	2022-05-21	West	Houston	furniture	Binders	1103.35	
9	ORD-10009	2022-03-02	Central	Dallas	furniture	Tables	1067.95	
19	ORD-10019	2022-09-11	West	Phoenix	Office Supplies	Phones	451.02	
20	ORD-10020	2022-10-14	South	Phoenix	Furniture	Chairs	925.54	
22	ORD-10022	2022-05-06	East	San Antonio	furniture	Phones	452.37	
...	...	...	...	...	...	...	...	
2030	ORD-10026	2022-10-02	Central	Chicago	furniture	Chairs	315.52	
2035	ORD-10528	2022-08-21	east	San Jose	furniture	Paper	103.44	
2042	ORD-11636	2022-01-21	WEST	New York	furniture	Phones	610.99	
2046	ORD-10282	2022-10-28	South	Los Angeles	Furniture	Machines	575.86	
2048	ORD-11279	2022-05-24	East	NaN	Technology	Accessories	758.35	

521 rows × 10 columns

Handle Missing Values (Numerical Columns)

For numerical columns like:

- Sales
- Profit
- Discount

We fill missing values using mean ,median.

In [296…

```
df["Sales"] = df["Sales"].fillna(df["Sales"].mean())
```

In [297…

```
df["Profit"] = df["Profit"].fillna(df["Profit"].mean())
```

In [298…
```
df["Discount"] = df["Discount"].fillna(df["Discount"].mean())
```

Handle Missing Values (Categorical Columns)

For columns like City.

- Fill with most common value.

In [299…
```
df["City"] = df['City'].fillna(df['City'].mode()[0])
```

In [300…
```
df.isnull().sum()
```

Out[300…
```
Order_ID        0
Order_Date      0
Region          0
City            0
Category        0
Sub_Category    0
Sales           0
Quantity        0
Discount        0
Profit          0
dtype: int64
```

Check Duplicate Rows

In [301…
```
df.duplicated().sum()
```

Out[301…
```
np.int64(50)
```

Remove Duplicate Rows

In [302…
```
df = df.drop_duplicates()
```

In [303…
```
# Verify again:

df.duplicated().sum()
```

Out[303…
```
np.int64(0)
```

Fix Inconsistent Region Names

- Dataset contains values like:

East east WEST West

In [304…
```
df['Region'] = df['Region'].str.title()
```

```
C:\Users\ADITYA\AppData\Local\Temp\ipykernel_11864\2754127899.py:1: SettingWithCo
pyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['Region'] = df['Region'].str.title()
```

In [305…

```
# Now check unique values.

df['Region'].unique()
```

Out[305…

```
array(['South', 'West', 'East', 'Central'], dtype=object)
```

Fix Category Column

- Some rows contain:

Furniture

furniture

In [371…

```
df['Category'] = df['Category'].str.title()
```

Convert Order Date to Datetime

Currently Order_Date is string.

In [372…

```
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
```

In [308…

```
df.dtypes
```

Out[308…

```
Order_ID                object
Order_Date      datetime64[ns]
Region                  object
City                    object
Category                object
Sub_Category            object
Sales                  float64
Quantity                 int64
Discount               float64
Profit                 float64
dtype: object
```

Extract Year and Month

In [373…

```
df['Year'] = df['Order_Date'].dt.year
df['Month'] = df['Order_Date'].dt.month
```

Final Check

In [310…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2000 entries, 0 to 1999
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Order_ID      2000 non-null   object
 1   Order_Date    2000 non-null   datetime64[ns]
 2   Region        2000 non-null   object
 3   City          2000 non-null   object
 4   Category      2000 non-null   object
 5   Sub_Category  2000 non-null   object
 6   Sales         2000 non-null   float64
 7   Quantity      2000 non-null   int64
 8   Discount      2000 non-null   float64
 9   Profit        2000 non-null   float64
 10  Year          2000 non-null   int32
 11  Month         2000 non-null   int32
dtypes: datetime64[ns](1), float64(3), int32(2), int64(1), object(5)
memory usage: 187.5+ KB
```

Now dataset should be:

Clean

- No missing values
- Correct data types
- Consistent categories

Insight

After cleaning the dataset:

- Missing values were successfully handled.
- Duplicate records were removed.
- Inconsistent text values were standardized.
- Date columns were converted into proper datetime format.
- The dataset is now ready for data analysis and feature exploration.

Why Data Cleaning Is Important

Real-world datasets contain:

- Missing values
- Duplicate records
- Incorrect formatting

Data cleaning ensures:

- Accurate analysis
- Reliable insights
- Better machine learning performance

Step 3 : Data Selection & Filtering Using Pandas

Select Single Column

- Returns the Sales column as a Pandas Series.

In [311…
```
df ["Sales"]
```

Out[311…
```
0        574.32
1       1427.06
2       1103.35
3        906.01
4        250.91
         ...
1995     992.29
1996    1435.79
1997     122.06
1998     104.44
1999     437.64
Name: Sales, Length: 2000, dtype: float64
```

Select Multiple Columns

- Returns a DataFrame with selected columns.

In [312…
```
df[['Sales', 'Profit', 'Region']]
```

Out[312…

	Sales	Profit	Region
0	574.32	90.93	South
1	1427.06	468.06	South
2	1103.35	58.24	West
3	906.01	247.61	East
4	250.91	-9.41	East
...	...	...	...
1995	992.29	160.25	South
1996	1435.79	41.40	West
1997	122.06	12.64	East
1998	104.44	3.03	South
1999	437.64	3.42	East

2000 rows × 3 columns

Select First Few Rows

In [313…

```
df.head(10)
```

Out[313...

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quantity
0	ORD-10000	2022-07-14	South	Houston	Office Supplies	Binders	574.32	
1	ORD-10001	2022-06-07	South	Houston	Technology	Accessories	1427.06	
2	ORD-10002	2022-05-21	West	Houston	Furniture	Binders	1103.35	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
4	ORD-10004	2022-01-07	East	San Antonio	Office Supplies	Accessories	250.91	
5	ORD-10005	2022-02-23	East	San Jose	Office Supplies	Storage	250.87	
6	ORD-10006	2022-04-14	Central	Houston	Technology	Tables	105.96	
7	ORD-10007	2022-12-08	East	San Antonio	Office Supplies	Phones	1301.94	
8	ORD-10008	2022-12-04	West	San Jose	Furniture	Chairs	909.65	
9	ORD-10009	2022-03-02	Central	Dallas	Furniture	Tables	1067.95	

Select Specific Rows Using Index

In [314...

```
df[0:6]
```

Out[314...

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quantity
0	ORD-10000	2022-07-14	South	Houston	Office Supplies	Binders	574.32	
1	ORD-10001	2022-06-07	South	Houston	Technology	Accessories	1427.06	
2	ORD-10002	2022-05-21	West	Houston	Furniture	Binders	1103.35	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
4	ORD-10004	2022-01-07	East	San Antonio	Office Supplies	Accessories	250.91	
5	ORD-10005	2022-02-23	East	San Jose	Office Supplies	Storage	250.87	

Select Data Using loc

- loc selects data by label (row name and column name).

In [315…
```
df.loc[0:5, ['Sales', 'Profit']]
```

Out[315…

	Sales	Profit
0	574.32	90.93
1	1427.06	468.06
2	1103.35	58.24
3	906.01	247.61
4	250.91	-9.41
5	250.87	75.35

Select Data Using iloc

- iloc selects data by index position.

In [316…
```
df.iloc[0:5, 3:6]
```

Out[316…

	City	Category	Sub_Category
0	Houston	Office Supplies	Binders
1	Houston	Technology	Accessories
2	Houston	Furniture	Binders
3	New York	Furniture	Binders
4	San Antonio	Office Supplies	Accessories

Filter Rows Based on Condition

- Example: Sales greater than 1000

In [317…
```
df[df['Sales'] > 1000]
```

Out[317…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
1	ORD-10001	2022-06-07	South	Houston	Technology	Accessories	1427.06	
2	ORD-10002	2022-05-21	West	Houston	Furniture	Binders	1103.35	
7	ORD-10007	2022-12-08	East	San Antonio	Office Supplies	Phones	1301.94	
9	ORD-10009	2022-03-02	Central	Dallas	Furniture	Tables	1067.95	
11	ORD-10011	2022-09-15	Central	Dallas	Technology	Chairs	1455.47	
...	...	...	...	...	...	...	...	
1986	ORD-11986	2022-12-08	East	New York	Furniture	Tables	1097.35	
1989	ORD-11989	2022-01-06	East	San Antonio	Furniture	Storage	1099.67	
1990	ORD-11990	2022-11-05	West	San Antonio	Furniture	Paper	1152.96	
1994	ORD-11994	2022-03-14	East	San Jose	Furniture	Tables	1132.04	
1996	ORD-11996	2022-08-20	West	Chicago	Office Supplies	Accessories	1435.79	

655 rows × 12 columns

Filter by Category

- Example: Only Furniture sales

In [318…

```
df[df['Category'] == 'Furniture']
```

Out[318…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quant
2	ORD-10002	2022-05-21	West	Houston	Furniture	Binders	1103.35	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
8	ORD-10008	2022-12-04	West	San Jose	Furniture	Chairs	909.65	
9	ORD-10009	2022-03-02	Central	Dallas	Furniture	Tables	1067.95	
15	ORD-10015	2022-01-31	West	San Jose	Furniture	Storage	291.44	
...	...	...	...	...	...	...	...	
1990	ORD-11990	2022-11-05	West	San Antonio	Furniture	Paper	1152.96	
1994	ORD-11994	2022-03-14	East	San Jose	Furniture	Tables	1132.04	
1995	ORD-11995	2022-06-23	South	Chicago	Furniture	Machines	992.29	
1998	ORD-11998	2022-12-28	South	San Antonio	Furniture	Tables	104.44	
1999	ORD-11999	2022-08-10	East	San Antonio	Furniture	Phones	437.64	

979 rows × 12 columns

Filter Multiple Conditions

Example:

- Sales > 500
- Profit > 100

In [319…

```
df[(df['Sales'] > 500) & (df['Profit'] > 100)]
```

Out[319…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
1	ORD-10001	2022-06-07	South	Houston	Technology	Accessories	1427.06	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
11	ORD-10011	2022-09-15	Central	Dallas	Technology	Chairs	1455.47	
18	ORD-10018	2022-01-11	South	Houston	Furniture	Phones	659.28	
23	ORD-10023	2022-02-16	West	San Jose	Furniture	Chairs	562.22	
...	...	...	...	...	...	...	...	
1986	ORD-11986	2022-12-08	East	New York	Furniture	Tables	1097.35	
1989	ORD-11989	2022-01-06	East	San Antonio	Furniture	Storage	1099.67	
1992	ORD-11992	2022-12-12	West	San Antonio	Office Supplies	Accessories	923.13	
1994	ORD-11994	2022-03-14	East	San Jose	Furniture	Tables	1132.04	
1995	ORD-11995	2022-06-23	South	Chicago	Furniture	Machines	992.29	

773 rows × 12 columns

OR Condition

- Example:

Region East OR West

In [320…

```
df[(df['Region'] == 'East') | (df['Region'] == 'West')]
```

Out[320…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quant
2	ORD-10002	2022-05-21	West	Houston	Furniture	Binders	1103.35	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
4	ORD-10004	2022-01-07	East	San Antonio	Office Supplies	Accessories	250.91	
5	ORD-10005	2022-02-23	East	San Jose	Office Supplies	Storage	250.87	
7	ORD-10007	2022-12-08	East	San Antonio	Office Supplies	Phones	1301.94	
...	...	...	...	...	...	...	...	
1993	ORD-11993	2022-09-18	West	Los Angeles	Office Supplies	Machines	220.32	
1994	ORD-11994	2022-03-14	East	San Jose	Furniture	Tables	1132.04	
1996	ORD-11996	2022-08-20	West	Chicago	Office Supplies	Accessories	1435.79	
1997	ORD-11997	2022-09-27	East	Los Angeles	Office Supplies	Tables	122.06	
1999	ORD-11999	2022-08-10	East	San Antonio	Furniture	Phones	437.64	

1357 rows × 12 columns

Filter Using isin()

- Example: Select multiple regions.

In [321…

```
df[df['Region'].isin(['East', 'West'])]
```

Out[321…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quant
2	ORD-10002	2022-05-21	West	Houston	Furniture	Binders	1103.35	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
4	ORD-10004	2022-01-07	East	San Antonio	Office Supplies	Accessories	250.91	
5	ORD-10005	2022-02-23	East	San Jose	Office Supplies	Storage	250.87	
7	ORD-10007	2022-12-08	East	San Antonio	Office Supplies	Phones	1301.94	
...	...	...	...	...	...	...	...	
1993	ORD-11993	2022-09-18	West	Los Angeles	Office Supplies	Machines	220.32	
1994	ORD-11994	2022-03-14	East	San Jose	Furniture	Tables	1132.04	
1996	ORD-11996	2022-08-20	West	Chicago	Office Supplies	Accessories	1435.79	
1997	ORD-11997	2022-09-27	East	Los Angeles	Office Supplies	Tables	122.06	
1999	ORD-11999	2022-08-10	East	San Antonio	Furniture	Phones	437.64	

1357 rows × 12 columns

Filter Using between()

- Example: Sales between 500 and 1000

In [322…

```
df[df['Sales'].between(500, 1000)]
```

Out[322…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Quan
0	ORD-10000	2022-07-14	South	Houston	Office Supplies	Binders	574.32	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
8	ORD-10008	2022-12-04	West	San Jose	Furniture	Chairs	909.65	
17	ORD-10017	2022-03-21	South	Phoenix	Office Supplies	Tables	796.64	
18	ORD-10018	2022-01-11	South	Houston	Furniture	Phones	659.28	
...	...	...	...	...	...	...	...	
1983	ORD-11983	2022-08-01	South	Phoenix	Furniture	Storage	734.79	
1987	ORD-11987	2022-01-12	West	Houston	Technology	Machines	754.76	
1988	ORD-11988	2022-07-11	Central	Los Angeles	Furniture	Paper	666.54	
1992	ORD-11992	2022-12-12	West	San Antonio	Office Supplies	Accessories	923.13	
1995	ORD-11995	2022-06-23	South	Chicago	Furniture	Machines	992.29	

706 rows × 12 columns

Filter Using query()

In [323…

```
df.query("Sales > 800 and Profit > 100")
```

Out[323…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
1	ORD-10001	2022-06-07	South	Houston	Technology	Accessories	1427.06	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
11	ORD-10011	2022-09-15	Central	Dallas	Technology	Chairs	1455.47	
28	ORD-10028	2022-05-03	West	New York	Office Supplies	Chairs	896.77	
30	ORD-10030	2022-04-17	West	Houston	Technology	Machines	919.17	
...	...	...	...	...	...	...	...	
1986	ORD-11986	2022-12-08	East	New York	Furniture	Tables	1097.35	
1989	ORD-11989	2022-01-06	East	San Antonio	Furniture	Storage	1099.67	
1992	ORD-11992	2022-12-12	West	San Antonio	Office Supplies	Accessories	923.13	
1994	ORD-11994	2022-03-14	East	San Jose	Furniture	Tables	1132.04	
1995	ORD-11995	2022-06-23	South	Chicago	Furniture	Machines	992.29	

568 rows × 12 columns

STEP 4 — Sorting & Data Manipulation Using Pandas

Sort Data by Sales (Ascending)

In [324…

```
df.sort_values(by='Sales')
```

Out[324…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
1921	ORD-11921	2022-11-03	West	Dallas	Furniture	Phones	24.76	
821	ORD-10821	2022-05-08	East	Los Angeles	Furniture	Tables	26.86	
1426	ORD-11426	2022-03-18	West	Houston	Technology	Phones	27.31	
208	ORD-10208	2022-05-07	South	New York	Office Supplies	Chairs	27.49	
1901	ORD-11901	2022-03-01	Central	San Antonio	Technology	Phones	27.74	
...	...	...	...	...	...	...	...	
1209	ORD-11209	2022-05-01	South	New York	Furniture	Phones	1495.74	
1054	ORD-11054	2022-03-02	West	Los Angeles	Furniture	Accessories	1496.94	
1954	ORD-11954	2022-11-21	West	Los Angeles	Technology	Phones	1497.55	
1464	ORD-11464	2022-02-06	Central	Chicago	Furniture	Phones	1499.13	
531	ORD-10531	2022-06-19	East	San Jose	Furniture	Machines	1499.58	

2000 rows × 12 columns

Explanation:

- Sorts dataset from lowest sales → highest sales.

In [325…

```
df.sort_values(by='Sales', ascending=False)
```

Out[325…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
531	ORD-10531	2022-06-19	East	San Jose	Furniture	Machines	1499.58	
1464	ORD-11464	2022-02-06	Central	Chicago	Furniture	Phones	1499.13	
1954	ORD-11954	2022-11-21	West	Los Angeles	Technology	Phones	1497.55	
1054	ORD-11054	2022-03-02	West	Los Angeles	Furniture	Accessories	1496.94	
1209	ORD-11209	2022-05-01	South	New York	Furniture	Phones	1495.74	
...	...	...	...	...	...	...	...	
1901	ORD-11901	2022-03-01	Central	San Antonio	Technology	Phones	27.74	
208	ORD-10208	2022-05-07	South	New York	Office Supplies	Chairs	27.49	
1426	ORD-11426	2022-03-18	West	Houston	Technology	Phones	27.31	
821	ORD-10821	2022-05-08	East	Los Angeles	Furniture	Tables	26.86	
1921	ORD-11921	2022-11-03	West	Dallas	Furniture	Phones	24.76	

2000 rows × 12 columns

Explanation:

- Sorts dataset from highest sales → lowest sales.
- Useful to find top sales transactions.

Sort by Multiple Columns

In [326…

```
df.sort_values(by=['Region', 'Sales'])
```

Out[326…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
1901	ORD-11901	2022-03-01	Central	San Antonio	Technology	Phones	27.74	
407	ORD-10407	2022-07-26	Central	Houston	Office Supplies	Tables	36.04	
1264	ORD-11264	2022-07-21	Central	New York	Furniture	Storage	37.20	
332	ORD-10332	2022-11-24	Central	San Jose	Furniture	Machines	41.30	
563	ORD-10563	2022-07-21	Central	Houston	Technology	Machines	41.53	
...	...	...	...	...	...	...	...	
532	ORD-10532	2022-11-13	West	New York	Furniture	Phones	1495.02	
1727	ORD-11727	2022-09-16	West	Phoenix	Office Supplies	Storage	1495.11	
847	ORD-10847	2022-04-01	West	New York	Furniture	Phones	1495.37	
1054	ORD-11054	2022-03-02	West	Los Angeles	Furniture	Accessories	1496.94	
1954	ORD-11954	2022-11-21	West	Los Angeles	Technology	Phones	1497.55	

2000 rows × 12 columns

Explanation:

- First sorts by Region, then by Sales within each region.

Sort Index

In [327…

```
df.sort_index()
```

Out[327…

	Order_ID	Order_Date	Region	City	Category	Sub_Category	Sales	Qua
0	ORD-10000	2022-07-14	South	Houston	Office Supplies	Binders	574.32	
1	ORD-10001	2022-06-07	South	Houston	Technology	Accessories	1427.06	
2	ORD-10002	2022-05-21	West	Houston	Furniture	Binders	1103.35	
3	ORD-10003	2022-10-02	East	New York	Furniture	Binders	906.01	
4	ORD-10004	2022-01-07	East	San Antonio	Office Supplies	Accessories	250.91	
...	...	...	...	...	...	...	...	
1995	ORD-11995	2022-06-23	South	Chicago	Furniture	Machines	992.29	
1996	ORD-11996	2022-08-20	West	Chicago	Office Supplies	Accessories	1435.79	
1997	ORD-11997	2022-09-27	East	Los Angeles	Office Supplies	Tables	122.06	
1998	ORD-11998	2022-12-28	South	San Antonio	Furniture	Tables	104.44	
1999	ORD-11999	2022-08-10	East	San Antonio	Furniture	Phones	437.64	

2000 rows × 12 columns

Explanation:

- Sorts dataset by row index.

Create New Column (Revenue Per Item)

In [374…

```
df.loc[:, 'Revenue_per_item'] = df['Sales'] / df['Quantity']
```

Create Profit Margin Column

In [375…

```
df.loc[:,'Profit_Margin'] = df['Profit'] / df['Sales']
```

Rename Columns

In [376…

```
df.rename(columns={'Sub_Category':'SubCategory'}, inplace=True)
```

Drop Column

In [377…

```
df.drop('Revenue_per_item', axis=1, inplace=True)
```

Select Specific Columns Order

In [332…

```
df = df[['Order_ID','Order_Date','Region','City','Category','SubCategory','Sales
```

Explanation:

- Rearranges columns in desired order.

Apply Mathematical Operations

- Example: Increase profit by 10%

In [333…

```
df['Profit'] = df['Profit'] * 1.10
```

Round Numeric Values

In [334…

```
df['Sales'] = df['Sales'].round(2)
```

Explanation:

- Rounds values to 2 decimal places.

Insight

After manipulating the dataset:

- Additional features such as Profit Margin were created.
- Sorting helps identify top-performing sales transactions.
- Feature engineering improves data understanding and analysis.
- These transformations prepare the dataset for deeper analysis and aggregation.

STEP 5 — GroupBy & Aggregation

Total Sales by Region

In [335…

```
df.groupby('Region')['Sales'].sum()
```

Out[335…

```
Region
Central    216618.09
East       519174.09
South      250799.48
West       530378.35
Name: Sales, dtype: float64
```

Explanation:

- Groups dataset by Region
- Calculates total sales for each region

Average Profit by Category

In [336…

```
df.groupby('Category')['Profit'].mean()
```

Out[336…

```
Category
Furniture          123.148534
Office Supplies    124.730124
Technology         126.397377
Name: Profit, dtype: float64
```

Explanation:

- Calculates average profit for each product category.

Number of Orders by City

In [337…

```
df.groupby('City')['Order_ID'].count()
```

Out[337…

```
City
Chicago        231
Dallas         238
Houston        303
Los Angeles    252
New York       237
Phoenix        233
San Antonio    256
San Jose       250
Name: Order_ID, dtype: int64
```

Explanation:

- Counts number of orders per city.

Multiple Aggregations

- Example: Sales statistics by region.

In [338…

```
df.groupby('Region')['Sales'].agg(['sum','mean','max','min'])
```

Out[338…

Region	sum	mean	max	min
Central	216618.09	714.911188	1499.13	27.74
East	519174.09	776.044978	1499.58	26.86
South	250799.48	737.645529	1495.74	27.49
West	530378.35	770.898765	1497.55	24.76

Explanation:

Calculates:

- total sales
- average sales
- highest sale
- lowest sale

GroupBy Multiple Columns

- Example: Sales by Region and Category

In [339…

```
df.groupby(['Region','Category'])['Sales'].sum()
```

Out[339…

```
Region   Category
Central  Furniture          112334.94
         Office Supplies     47454.27
         Technology          56828.88
East     Furniture          270213.82
         Office Supplies    113835.44
         Technology         135124.83
South    Furniture          109751.09
         Office Supplies     69507.01
         Technology          71541.38
West     Furniture          261243.17
         Office Supplies    141650.00
         Technology         127485.18
Name: Sales, dtype: float64
```

Reset Index

- Sometimes grouped results look messy.

In [340…

```
df.groupby('Region')['Sales'].sum().reset_index()
```

Out[340…

	Region	Sales
0	Central	216618.09
1	East	519174.09
2	South	250799.48
3	West	530378.35

Sort Grouped Data

- Example: Highest sales region.

In [341…

```
df.groupby('Region')['Sales'].sum().sort_values(ascending=False)
```

Out[341…

```
Region
West       530378.35
East       519174.09
South      250799.48
Central    216618.09
Name: Sales, dtype: float64
```

Advanced Aggregation

In [342…

```
df.groupby('Category').agg({
    'Sales':'sum',
    'Profit':'mean',
    'Quantity':'sum'
})
```

Out[342…

	Sales	Profit	Quantity
Category			
Furniture	753543.02	123.148534	4954
Office Supplies	372446.72	124.730124	2490
Technology	390980.27	126.397377	2578

Insights

From the grouped analysis we can identify:

- The region generating highest sales
- The product category contributing highest profit
- Cities with maximum orders

This type of analysis helps businesses understand which areas or products drive revenue.

STEP 6 — Value Counts & Frequency Analysis

Count Occurrences of Each Category

In [343…

```
df['Category'].value_counts()
```

Out[343…

```
Category
Furniture          979
Technology         518
Office Supplies    503
Name: count, dtype: int64
```

Explanation:

- Counts how many times each product category appears.

Count Occurrences of Region

In [344…
```
df['Region'].value_counts()
```

Out[344…
```
Region
West       688
East       669
South      340
Central    303
Name: count, dtype: int64
```

- Shows number of orders from each region.

Count Occurrences of Cities

In [345…
```
df['City'].value_counts()
```

Out[345…
```
City
Houston        303
San Antonio    256
Los Angeles    252
San Jose       250
Dallas         238
New York       237
Phoenix        233
Chicago        231
Name: count, dtype: int64
```

Normalize Value Counts (Percentage)

In [346…
```
df['Region'].value_counts(normalize=True)*100
```

Out[346…
```
Region
West       34.40
East       33.45
South      17.00
Central    15.15
Name: proportion, dtype: float64
```

Explanation:

- Returns percentage distribution instead of counts.

Unique Values in Column

In [347…
```
df['Category'].unique()
```

Out[347…
```
array(['Office Supplies', 'Technology', 'Furniture'], dtype=object)
```

Count Number of Unique Values

In [348…
```
df['City'].nunique()
```

Out[348… 8

Frequency Table for Two Columns

In [349…
```
pd.crosstab(df['Region'], df['Category'])
```

Out[349…

Category	Furniture	Office Supplies	Technology
Region			
Central	150	68	85
East	345	150	174
South	152	98	90
West	332	187	169

- Shows relationship between two categorical columns.

Insights

From frequency analysis we can determine:

- Which product category appears most frequently
- Which region generates the highest number of orders
- Which cities have the highest sales activity
- Distribution of orders across categories and regions

This analysis helps businesses understand market demand and customer distribution.

STEP 7 — Pivot Tables in Pandas

Basic Pivot Table

Example: Total Sales by Region

In [350…
```
pd.pivot_table(df, values='Sales', index='Region', aggfunc='sum')
```

Out[350…

	Sales
Region	
Central	216618.09
East	519174.09
South	250799.48
West	530378.35

Explanation:

- values → column to analyze
- index → grouping column
- aggfunc → aggregation function

Average Profit by Category

In [351…

```
pd.pivot_table(df, values='Profit', index='Category', aggfunc='mean')
```

Out[351…

Category	Profit
Furniture	123.148534
Office Supplies	124.730124
Technology	126.397377

Pivot Table with Multiple Aggregations

In [352…

```
pd.pivot_table(df, values='Sales', index='Region', aggfunc=['sum','mean','max'])
```

Out[352…

	sum	mean	max
	Sales	Sales	Sales
Region			
Central	216618.09	714.911188	1499.13
East	519174.09	776.044978	1499.58
South	250799.48	737.645529	1495.74
West	530378.35	770.898765	1497.55

Pivot Table with Multiple Index Columns

Example: Sales by Region and Category

In [353…

```
pd.pivot_table(df, values='Sales', index=['Region','Category'], aggfunc='sum')
```

Out[353…

Region	Category	Sales
Central	Furniture	112334.94
	Office Supplies	47454.27
	Technology	56828.88
East	Furniture	270213.82
	Office Supplies	113835.44
	Technology	135124.83
South	Furniture	109751.09
	Office Supplies	69507.01
	Technology	71541.38
West	Furniture	261243.17
	Office Supplies	141650.00
	Technology	127485.18

Pivot Table with Columns

Example: Sales by Region and Category

In [354…

```
pd.pivot_table(df, values='Sales', index='Region', columns='Category', aggfunc='
```

Out[354…

Category	Furniture	Office Supplies	Technology
Region			
Central	112334.94	47454.27	56828.88
East	270213.82	113835.44	135124.83
South	109751.09	69507.01	71541.38
West	261243.17	141650.00	127485.18

Pivot Table with Multiple Values

In [355…

```
pd.pivot_table(df,
               values=['Sales','Profit'],
               index='Region',
               aggfunc='sum')
```

Out[355…

Region	Profit	Sales
Central	34437.152270	216618.09
East	87315.690175	519174.09
South	43955.629027	250799.48
West	83067.037662	530378.35

Fill Missing Values in Pivot Table

In [356…

```
pd.pivot_table(df,
               values='Sales',
               index='Region',
               columns='Category',
               aggfunc='sum',
               fill_value=0)
```

Out[356…

Category	Furniture	Office Supplies	Technology
Region			
Central	112334.94	47454.27	56828.88
East	270213.82	113835.44	135124.83
South	109751.09	69507.01	71541.38
West	261243.17	141650.00	127485.18

Insights

Using pivot tables we can easily identify:

- Which region contributes most sales
- Which category performs best in each region
- Overall sales and profit distribution
- Cross-analysis between region and product categories

Pivot tables allow fast multi-dimensional data analysis.

STEP 8 — Apply & Lambda Functions in Pandas

Using apply() on a Column

Example: Increase Sales by 5%

In [357…

```
df['Updated_Sales'] = df['Sales'].apply(lambda x: x * 1.05)
```

Explanation:

- apply() → applies a function to each value
- lambda → small anonymous function
- x → each value in Sales column

Create Profit Category

Example: Categorize profit as High or Low.

In [358…

```
df['Profit_Category'] = df['Profit'].apply(lambda x: 'High Profit' if x > 200 el
```

Apply Function on Multiple Columns

Example: Calculate Net Revenue

In [359…

```
df['Net_Revenue'] = df.apply(lambda row: row['Sales'] - row['Discount'], axis=1)
```

Explanation:

- axis=1 → row-wise operation
- Uses multiple columns.

Use apply() for String Operations

Example: Convert City names to uppercase.

In [360…

```
df['City'] = df['City'].apply(lambda x: x.upper())
```

Create Discount Category

In [361…

```
df['Discount_Level'] = df['Discount'].apply(
    lambda x: 'High Discount' if x > 0.2 else 'Low Discount'
)
```

Apply Mathematical Transformation

Example: Round Sales values.

In [362…

```
df['Sales'] = df['Sales'].apply(lambda x: round(x,2))
```

Use apply() with Conditional Logic

Example: Identify large orders.

In [363…

```
df['Large_Order'] = df['Quantity'].apply(lambda x: 'Yes' if x >= 5 else 'No')
```

Apply Function to Entire DataFrame

Example: Find maximum values per column.

In [364…

```
df.apply(np.max)
```

Out[364…

```
Order_ID                     ORD-11999
Order_Date         2022-12-31 00:00:00
Region                            West
City                          SAN JOSE
Category                    Technology
SubCategory                     Tables
Sales                          1499.58
Quantity                             9
Discount                           0.3
Profit                         736.296
Updated_Sales                 1574.559
Profit_Category             Low Profit
Net_Revenue                    1499.38
Discount_Level            Low Discount
Large_Order                        Yes
dtype: object
```

Replace Values

Example: Replace category name.

In [365…

```
df['Category'] = df['Category'].replace('Office Supplies','Office')
```

Insights

Using apply() and lambda functions allows flexible data transformation. These operations help create new features such as profit categories, discount levels, and large order indicators, which can be useful for deeper business analysis and machine learning models.

STEP 9 — Business Insights

In this section we analyze the dataset and extract meaningful insights.

Region Generating Highest Sales

In [366…

```
df.groupby('Region')['Sales'].sum().sort_values(ascending=False)
```

Out[366…

```
Region
West       530378.35
East       519174.09
South      250799.48
Central    216618.09
Name: Sales, dtype: float64
```

Insight Example:

- The West region generates the highest total sales compared to other regions. This indicates strong market demand in this region.

Most Profitable Category

In [367…

```
df['City'].value_counts().head(5)
```

Out[367…

```
City
HOUSTON        303
SAN ANTONIO    256
LOS ANGELES    252
SAN JOSE       250
DALLAS         238
Name: count, dtype: int64
```

Insight Example:

- Cities such as HOUSTON and SAN ANTONIO have the highest number of orders, indicating high customer demand in these locations.

Sales Performance by Category

In [368…

```
df.groupby('Category')['Sales'].sum()
```

Out[368…

```
Category
Furniture     753543.02
Office        372446.72
Technology    390980.27
Name: Sales, dtype: float64
```

Insight Example:

- Furniture and Technology categories contribute the largest share of sales revenue.

Profit by Region

In [369…

```
df.groupby('Region')['Profit'].sum()
```

Out[369…

```
Region
Central    34437.152270
East       87315.690175
South      43955.629027
West       83067.037662
Name: Profit, dtype: float64
```

Insight Example:

- Certain regions generate higher profit margins, suggesting better market performance and customer purchasing power.

Average Discount by Category

In [370…

```
df.groupby('Category')['Discount'].mean()
```

Out[370…

```
Category
Furniture     0.147185
Office        0.144725
Technology    0.161673
Name: Discount, dtype: float64
```

Insight Example:

- Some product categories receive higher average discounts, which may impact profitability.

Overall Insights Summary

From the analysis we observe:

- The West region contributes the highest sales revenue.
- Technology products generate the highest profit.
- Major cities have high sales activity.
- Discount strategies vary across product categories and may affect profit margins.

These insights help businesses understand which markets and products drive revenue.

Recommendations

Based on the analysis, the following recommendations can be made:

- The company should focus on high-performing regions such as the West to maximize revenue growth.
- Since Technology products generate high profits, the company should expand product offerings in this category.
- Cities with high order volumes should be targeted with marketing campaigns and promotions.
- Discount strategies should be carefully monitored to ensure they do not significantly reduce profitability.
- Data-driven analysis like this should be performed regularly to support better business decision-making.

Conclusion

In this project, we used Pandas for data analysis and data manipulation on a retail sales dataset. The dataset contained issues such as missing values, duplicate records, and inconsistent data formats, which were cleaned using Pandas data cleaning techniques.

After preparing the dataset, various analytical techniques such as groupby operations, pivot tables, filtering, and aggregation were applied to explore the data and extract meaningful insights. The analysis helped identify key sales trends, profitable product categories, and regional sales performance.

This project demonstrates how Pandas can be effectively used to clean, manipulate, and analyze real-world datasets for business decision-making.